LiDAR × 模倣学習による TurtleBot3 自律走行

Physical AI 応用1講座 最終課題 説明資料 ／ ROS 2 Humble・Gazebo Classic 11・TurtleBot3 Burger・PyTorch
コード: https://github.com/2life2021/tb3-ai-navigator ／ 動画: YouTube（omnicampusで提出）

1. 概要と目的

TurtleBot3 が LiDAR（`/scan`）の距離情報だけを使い、**ニューラルネットワーク（MLP）の推論結果**で「前進・左旋回・右旋回」を選び、`/cmd_vel` に速度指令を出して、障害物のある環境（turtlebot3_world）を衝突せずに走り続けることをタスクとしました。

if文で直接ロボットを動かすのではなく、**ルールで書いた「熟練者（教師）」の振る舞いを、シミュレーション内で自動収集したデータから学習（模倣学習）**させ、走行中は学習済みモデルだけで判断させます。さらに、本当に動くかを**閉ループ（ロボットを実際に走らせて）で定量評価し、失敗を分析して改善するサイクル**を回したことが本作品の中心です。

実世界では、掃除ロボットや倉庫の搬送ロボットの「ぶつからずに動き続ける」という基本機能に相当します。ルールを人手で調整し続けるかわりに、データからセンサ→行動の対応を学ぶ構成にしておくと、人間の操作ログや強化学習など、教師を別のものに差し替えて発展させることができます。

2. システム構成

```
Gazebo（turtlebot3_world）
  ｜ /scan  LaserScan 360本, 約5Hz
  ▼
ai_driver.py（ROS 2 ノード）
  前処理: inf→3.5m, 5° ごと72セクタの最小距離, ÷3.5 で 0〜1 に正規化
  MLP: 72 → 128 → 128 → 3（ReLU, softmax） ……  P(前進), P(左), P(右)
  制御: argmax（最大確率の行動）  または  soft（確率で速度を重み付け平均）
  ｜ /cmd_vel  Twist   前進 v=0.18m/s ／ 左右 v=0.02m/s, ω=±0.9rad/s
  ▼
Gazebo 上の TurtleBot3 Burger          ai_monitor.py: LiDARと確率を可視化（制御には関与しない）
```

3. AIモデル（入力・出力・学習方法）

項目	内容
モデル	多層パーセプトロン（PyTorch）。入力72 → 隠れ層128×2（ReLU） → 出力3
入力	LiDAR 360本を5°ごとの72セクタに分け、各セクタの最小距離を0〜1に正規化したベクトル
出力	3行動（FORWARD / LEFT / RIGHT）の確率
学習方法	模倣学習（Behavior Cloning）＝教師あり3クラス分類。クラス重み付き交差エントロピー、Adam
学習データ	Gazebo で自動走行しながら収集した（生LiDAR 360本, 行動ラベル）3,206組。学習用（2,565組）は左右反転とノイズ付加で25,650組に拡張
正解ラベル	**ルールベースの教師（teacher_action_v2）で自動生成。**前方±30°が0.55m以上かつ斜め前30〜75°が0.35m以上空いていれば前進（ただし真横75〜105°に0.22m未満の壁があればその壁から離れる方向へ旋回）。それ以外は、左右20〜100°で平均距離が大きい方へ旋回
推論	`/scan` を受信するたびに CPU で1回推論（約5Hz）。走行中の行動は MLP の出力だけで決める

> **AIの役割と限界**

> - 走行中の判断は MLP のみが行います。教師ルールは、評価指標（教師との一致率）を計算するためだけに呼び出しています。
> - 正解ラベルがルール由来なので、性能の上限は基本的に教師と同程度です。「AIが教師より賢くなる」ことは主張しません。
> - 地図や自己位置推定を持たない反応型の方策なので、ゴールへの移動や探索はできません。学習した環境以外での性能は検証していません。

4. 開発の経緯と工夫した点

① まず最小構成を動かす（v1）

教師v1でデータ1,176件を収集し、24セクタ（15°刻み）のMLPを学習しました（検証精度91%）。120秒の走行で衝突0回、18.95m を走行し、合格ラインの動作を確認しました。

② DAgger を試したら悪化 → ログを解析して原因の仮説を立てる

模倣学習の弱点（AIが自分で操作すると、学習データにない状態に入ってしまう＝共変量シフト）への対策として、DAggerを導入しました。AI自身が走ったときの状態に教師ラベルを付けて追加学習する手法です。ところが、検証精度は91.1%→93.9%に上がったのに、初期位置をそろえた120秒の走行で**衝突イベントが0回から6回に増えました**（各1回）。

観測事実: 6回のうち4回は左下の柱付近、2回は外周の壁付近で起きました。直前のログには、前進確率≈1.0のまま最小距離が縮んでいく場面と、その場旋回中に最小距離が縮む場面がありました。また、教師v1は前方±60°しか見ておらず、真横の障害物を判断に使っていません（コードで確認できる事実）。
仮説（未検証）: 教師v1が「横の障害物に近い状態での前進」を正解としていたため、DAggerで障害物近くの状態データが増えたことで、その判断が強化された。ただし、ログには最小距離の方向を記録していないためこの仮説は直接確かめられず、1回ずつの比較なので偶然の可能性も残ります。
この分析から「モデルよりも教師（ラベル）の質を疑うべき」と判断し、真横の確認を加えた教師v2を作りました。

③ 生データを保存して再ラベル付け

収集時に生LiDAR値も保存していたので、教師を改良しても**再走行せずに全データへラベルを付け直せます**（`train.py --relabel`）。

④ 特徴量の角度分解能とデータ拡張

15°刻みのセクタでは、教師の角度境界（30°・75°・105°）とセクタ境界がずれるため、5°刻み72セクタに変更しました。さらに、左右反転と距離ノイズ（σ=2cm）を加えたスキャンを教師で再ラベルして、学習データを約10倍に拡張しました（検証は拡張していない元データで行う）。

⑤ 確率を使った soft 制御と可視化

最大確率の行動を選ぶ方式（argmax）に加えて、行動確率で速度を重み付け平均する soft 制御も実装しました。AIの確信度が低い（確率が割れている）ときは自動的に減速し、動きも滑らかになります。また、AIが見ているLiDARと出力確率をリアルタイムに表示するモニターを作り、動画で「AIが何を見て、どう判断したか」がわかるようにしました。

5. 評価

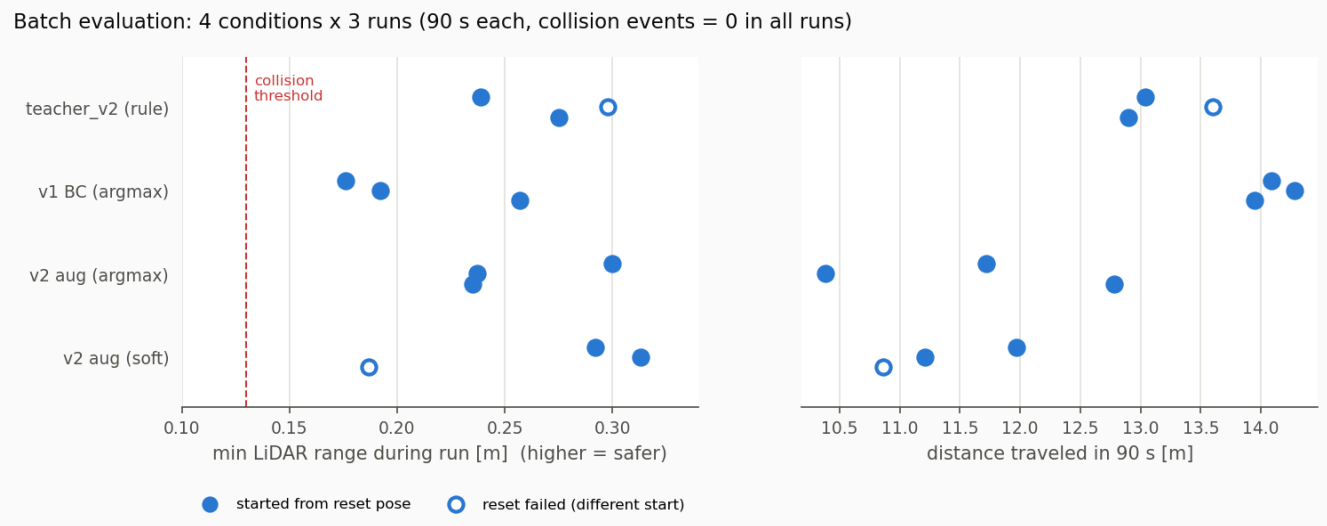
各試行の開始時に `/reset_world` を呼んで初期位置に戻し（12回中2回は失敗。後述の監査を参照）、各条件を複数回走らせて自動記録しました。

評価指標: 走行距離＝/odom の積算値。最接近距離＝走行中の LiDAR 最小値（大きいほど安全）。衝突イベント＝LiDAR最小距離が0.13m未満になった回数（接触センサではない）。教師一致率＝AIの行動が教師ルールと一致した割合。

一括評価（4条件 × 3回 × 90秒）

条件	試行	走行距離 [m]	最接近距離 [m] 平均（最小）	衝突イベント	教師一致率（対v2）
教師v2（ルールで走行）	3	13.2 ± 0.3	0.271 (0.239)	0	1.00
v1 BC (argmax)	3	14.1 ± 0.1	0.208 (0.176)	0	0.79 ※1
v2 aug (argmax)［最終採用］	3	11.6 ± 1.0	0.257 (0.235)	0	0.80
v2 aug (soft)	3	11.3 ± 0.5	0.264 (0.187)	0	0.83

※1 v1は教師v1で学習したモデルなので、教師v2との一致率は参考値です。※2 教師v2の2回目とsoftの1回目は `/reset_world` が効かず、別の位置から始まっています（図の白抜きの点）。初期位置が正しかった回だけで集計すると、教師v2は最接近距離の平均0.257m（最小0.239m）、softは平均0.303m（最小0.292m）です。



全12回とも衝突イベントは0回でした。最終採用モデルは、v1に比べて障害物との距離を大きく保ちます（最接近距離の最小値 0.176m→0.235m）。その代わり旋回が増えて、走行距離は短くなりました（14.1m→11.6m）。事後分析では、全試行で0.25m四方のマスを36〜64個訪れており、立ち往生（5秒間の移動が0.10m未満）は最大4.4秒でした。

デモ動画の走行（最終採用モデル、150秒、リセット後の初期位置から開始）：走行距離17.8m、衝突イベント0回、最接近距離 **0.187m**。一括評価での最小値（0.235m）より障害物に近づいており、3回の評価だけでは最悪ケースを把握しきれていないことを示しています。

実験設計の監査（自己点検）

項目	確認結果

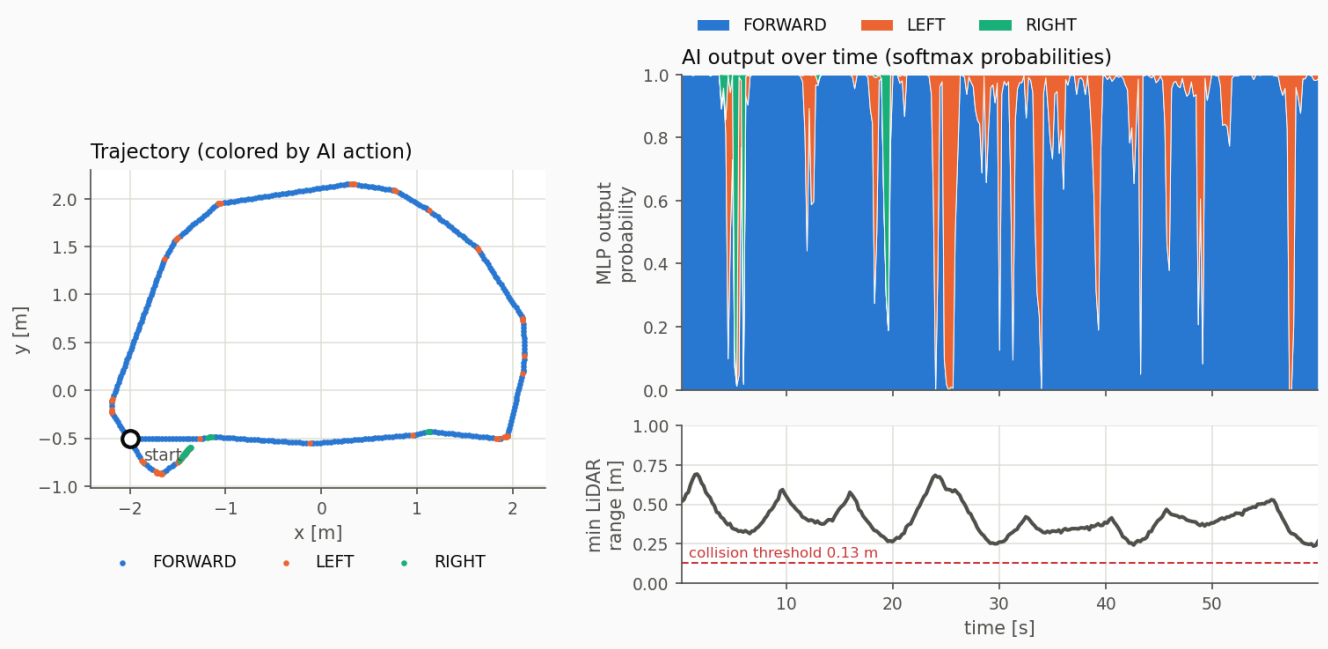
初期位置・向き	12回中10回は同じ位置から開始し、2回はリセットに失敗した。**コードがリセットの成否を確認していなかった。**向きはログに記録しておらず、検証できない
試行の多様性	同じ初期位置で3回なので、違いはLiDARノイズ（σ=1cm）とタイミングの揺らぎだけ。初期位置への依存性は評価できておらず、統計的な有意差も言えない
衝突の定義	LiDAR最小距離が0.13m未満になった回数。接触センサではないため、LiDARの死角での接触は検出できない
評価コードと提出コード	評価の後に追加したのは可視化用トピックの送信だけで、行動の決定と速度指令の計算は同一。提出コードでは短い動作確認だけを行った
改善案（未実施）	リセット後の位置確認と再試行、yawの記録、複数の初期位置×5回以上、接触センサによる判定、提出コードでの再評価

学習の検証精度

モデル	教師	特徴量	データ	検証精度
v1	v1	24セクタ	1,176	91.1%
v1 + DAgger	v1	24セクタ	2,039	93.9%
v2 (24セクタ)	v2	24セクタ	3,206（再ラベル）	87.1%
v2 (72セクタ)	v2	72セクタ	3,206	88.3%
v2 aug	v2	72セクタ	3,206＋拡張	87.2%

v1とv2では教師が異なるため、精度を直接比較することはできません。検証精度が最も高い「v1+DAgger」が走行では最も悪かったことから、**オフラインの精度だけでは走行性能を判断できず、閉ループ評価が不可欠**だとわかります。

走行の例（最終採用モデル、一括評価の1回目）



左: 軌跡（点の色＝AIが選んだ行動）。右上: MLPが出力した各行動の確率の時間変化（最初の60秒）。右下: LiDARの最小距離。

6. 再現性と開発の記録

詳細は GitHub の `docs/REPRODUCIBILITY.md` と `docs/DEVELOPMENT_LOG.md` にあります。

何が学習され、何を人間が決めたか

区分	値
学習される値	MLPの重みとバイアス（26,243個）**のみ**
ラベルの定義（教師の閾値）	v1: 前方±25°>0.55m、斜め25〜60°>0.30m ／ v2: 前方±30°>0.55m、斜め30〜75°>0.35m、真横75〜105°<0.22mなら反対側へ旋回
特徴量	セクタ数 24（v1系）／72（最終）、最小値÷3.5
行動の速度	前進 (0.18m/s, 0) ／ 左右 (0.02m/s, ±0.9rad/s)。全期間で変更なし
データ収集	ε（ランダム行動の確率、1〜4ステップ継続）、β（DAggerで教師が操作する確率）、収集時間（実時間）、0.13mでリセット
学習	隠れ層 64 / 128、エポック 300 / 600 / 150、バッチ128、Adam lr=1e-3、weight_decay=1e-4、検証20%、シード0、拡張 0 / 4
推論・評価	argmax / soft、scan受信ごとに推論、衝突判定 0.13m、試行 90 / 120 / 150秒（実時間）

データ収集時の教師の条件

データ	運転していたもの	保存ラベル	ε / β	時間	件数	前/左/右
teacher	教師v1	v1	0.10 / –	240s	1,176	1020/85/71
dagger1	v1モデル（確率0.3で教師v1）	v1	0.05 / 0.3	180s	863	721/84/58
teacher_v2	教師v2	v2	0.10 / –	240s	1,167	717/233/217

v2系のモデルは、3つのデータすべてを保存済みの生LiDARから**教師v2で再ラベル**して学習しました。つまり、**最終モデルの学習データには、DAggerで集めた状態（863件）が教師v2のラベルで含まれています。**悪化したのは「v1のラベルのまま追加学習した」条件であり、DAggerの状態データが最終モデルにどれだけ寄与したかは、切り分けて評価していません。

推論・制御の変更で起きた問題と修正（抜粋）

問題（観測）	原因	修正 → 結果
開始位置が試行ごとに違い、比較が不公平だった	確認済み: 各試行が前回の終了位置から始まっていた	`--reset` を追加 → 以後はほぼ同じ位置から開始（12回中2回は失敗）
`--reset` を付けると AttributeError で落ちた	確認済み: 初期化の途中でサービスの応答を待ったため、scan のコールバックが先に実行された	リセット処理を購読の登録より前に移した → 解消
v1+DAggerで衝突イベント6回（検証精度は最高）	推定: 教師v1が真横を見ず、障害物の近くでの前進を正解としていた	教師v2を作成 → 教師単体の最接近距離 0.198→0.234m
最初のv2モデルの検証精度が79%	推定: データ不足（約1,100件）とモデルの幅不足	全データを再ラベルし、幅128に → 87.1%
120秒の指定で290秒かかり、scanが約0.9Hzに低下	未解明（直後の計測では4.86Hzで正常）	推論を1スレッドに（予防策。効果は未検証）→ 以後は再発なし

モデルによって入力の次元（24/72）が異なる	設計上の問題: 特徴量を呼び出し側で計算していた	セクタ数をモデルに保存し、推論側で特徴量を計算 → 次元の取り違えを防止
argmaxでは迷っていても全速で動く	制御方式の性質	soft制御を追加 → 衝突0回で同程度。優劣は判断できず

再現性の評価

- **学習は再現できる**: 別々に学習した2つのモデル（同じデータ・設定・シード）で、学習精度0.98441・検証精度0.87051が完全に一致しました（この環境での確認のみ）。
- **データ収集と走行は完全には再現できない**: 乱数シードを固定しておらず、LiDARノイズや通信のタイミングでも軌跡が変わります。収集時の開始位置と、すべての試行のyawは記録していません。時間はシミュレーション時間ではなく実時間で区切っています。
- **記録から得た学び**: 検証精度が最も高かったモデル（v1+DAgger、93.9%）が走行では最も悪く、データ拡張は精度を上げなかったのに走行では障害物から距離を保ちました。オフラインの指標と閉ループの性能は別物です。また、開始位置、一致率の基準、シミュレーション速度など、気づかないうちに比較条件が変わる要因が多くありました。どれも、ログを残していたから後で気づけたものです。

7. 学び・限界と今後の課題

考察

- **模倣学習の質は教師（ラベル）の質に強く依存する**: DAggerの悪化は、手法そのものより教師ルールの欠陥（真横を見ていない）による可能性が高いと考えています。ただし、障害物の方向を記録していないためこれは未検証の仮説です。それでも、閉ループのログ解析がなければ、この仮説にたどり着くこともできませんでした。
- **検証精度は約87%で頭打ち**: 誤りの多くは前進と旋回の閾値付近で起きており、ラベル自体があいまいになる状態です。

制作を通して得た学び

- **開発環境は階層的なシステムである**: OSの中核であるカーネルがプロセスやメモリを管理し、シェル（PowerShell、Bash）はそれを操作するためのインターフェースです。Dockerのコンテナは独立したカーネルを持たず、ホスト側のカーネル（今回はWindows上の仮想化環境が提供するLinuxカーネル）を利用します。ROS 2はノード間の通信を担うミドルウェア、Gazeboは物理シミュレータで、OSやシェルとは異なる層に位置します。
- **ROS 2の通信とコールバック**: ノードは処理を担当する単位、トピック（`/scan`、`/cmd_vel`）はノード間の通信経路です。`self.on_scan`（メソッドの参照）を登録しておくと、データが届いたときに呼び出される、というコールバックの仕組みを理解しました。
- **障害物回避とナビゲーションは別の能力**: MLPへの入力は `/scan` のみで、`/odom`（誤差を含みうる位置推定）は使っていません。本システムは周囲の状況に反応して衝突を避ける反応型の制御であり、地図や自己位置を使って目的地に到達する能力とは異なります。
- **似た概念を区別する**: ニューロン数128や学習率は人間が決めるハイパーパラメータ、重みとバイアス（26,243個）は学習で調整されるモデルパラメータです。データ拡張（学習用2,565件→25,650件）で増やしたのは学習データであって、モデルの大きさではありません。softmaxの値は現実の成功確率を保証するものではありません。そして、**検証精度87.2%は教師ルールとの行動一致率であり、障害物回避の成功率ではありません。**

限界と改善案

	限界・課題	改善案
①	**未知環境への汎化**: 評価は特定の環境のみで行いました。学習データと検証データを同じ走行記録から分割しているため、検証精度が未知環境での性能を反映していないおそれがあります	学習用と評価用で障害物配置・初期位置を分離し、走行成功率、最短距離、停止・接触回数を比較する
②	**教師ルールへの依存**: 正解は人間が設計したルールで、最適とは限りません。MLPが教師より優れていることも確認できていません	教師の改善、人間の操縦データの利用、教師とMLPを同一条件で比較して利点を定量化する
③	**自己位置推定・経路計画がない**: 目的地への到達は保証できません。`/odom` を入力に加えるだけでは、正確な自己位置は得られません	SLAM・自己位置推定・経路計画を導入し、目的地への自律移動へ発展させる
④	**モデル構造・特徴量の妥当性**: 72-128-128が最適かは未確認です。5°ごとの最小距離への圧縮で失われる情報もあります	入力36/72/360次元 × 隠れ層64/128/256で、走行性能と推論速度を比較する
⑤	**評価指標・実験条件の不足**: 不一致の中には、危険な前進と不要な旋回のように重みの異なるものが混在しています。試行回数が少なく、リセットに失敗した試行もありました	初期条件と乱数を管理して試行を増やし、ばらつきと危険な誤分類を重点的に評価する
⑥	**実機適用と再現性**: 実機ではセンサー誤差、車輪の滑り、通信遅延などが生じます	ノイズや摩擦を変えて頑健性を確認し、段階的に実機試験へ移行する。環境構築や実験条件をGitHubで明示する

総括と今後の研究方針

最大の学びは、ロボットの自律走行を単一のAIの働きとしてではなく、**開発環境・通信・センシング・制御・機械学習が階層的に連携するシステム**として理解できたことです。特に、**何を入力し、何を正解として学習し、どのような指標で評価しているのかを正確に把握すること**が、機械学習システムの仕組みと限界を理解するうえで不可欠だと実感しました。

今後は、分類精度の向上そのものではなく、**安全な障害物回避という本来の目的に対して、機械学習がどの程度有効なのか**を検証したいと考えています。教師ルールとMLPの走行性能を同一条件で比較し、未知の障害物配置での汎化性能や、モデルの複雑さと制御性能の関係を調べます。機械学習を導入すること自体を目的とするのではなく、**どのような条件で、なぜ機械学習が有効になるのか**を明らかにすることを今後の課題とします。

8. 環境構築と実行方法

層	内容
ホスト	Windows 11。GPUは不使用
WSL2 / Docker	WSL2 上の **Docker Desktop 29.8.2**。コンテナは独自のカーネルを持たず、WSL2のLinuxカーネルを共有する
コンテナ	イメージ `airobotbook/ros2-desktop-ai-robot-book-humble`（約25GB）。Ubuntu 22.04.5、ROS 2 Humble、Gazebo 11.10.2、Python 3.10.12、PyTorch 2.6.0。**本作品のための追加インストールはなし**
ROS 2 Humble	イメージ内に **apt** で導入済み（`/opt/ros/humble`）。`~/.bashrc` が `/opt/ros/humble`（下地）→ `~/airobot_ws`（ソースからビルドしたワークスペースを上乗せ）の順に `source` して有効にする
画面	コンテナ内のVNCデスクトップを、ブラウザ（noVNC、http://localhost:6080）で表示
シミュレーション	`~/airobot_ws` の `turtlebot3_gazebo`（AI-Robot-Book-Humble/turtlebot3_happy_mini）、ワールド `turtlebot3_world`、ロボット TurtleBot3 Burger

```
docker run -d --name pai_ros2 -p 6080:80 --shm-size=512m -v "$(pwd)":/workspace ¥
  airobotbook/ros2-desktop-ai-robot-book-humble:latest          # コンテナ作成（初回のみ）
git clone https://github.com/2life2021/tb3-ai-navigator.git     # コンテナ内の /workspace で
export TURTLEBOT3_MODEL=burger                                    # 必須（.bashrc の既定は happy_mini）
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py         # noVNC のターミナルで
python3 scripts/ai_driver.py --model models/policy_v2_aug.pt --reset   # AI自律走行（teleopは停止しておく）
python3 scripts/ai_monitor.py                                          # AI視点モニター（任意）
```

詳しい手順・確認方法・よくある問題は `docs/SETUP.md`、データ収集・学習・一括評価の手順は README、各モデルを再学習するコマンドは `docs/REPRODUCIBILITY.md` を参照してください。